

- Self-replicating malware spreads through networks using user action.
- Viruses: Malicious code that attaches to legitimate programs and executes when the host runs.
- Trojan Horses: Program that looks harmless but secretly performs malicious operations.
- Spyware: Malware that secretly collects user activity or personal data without consent.
- Ransomware: Malware that locks or encrypts your files and then demands payment (a ransom) to unlock them.
- Botnets: Network of infected devices remotely controlled by an attacker.
- Phishing: Fraudulent attempt to obtain sensitive information by impersonating trusted entities (often via email or fake websites).
- Pharming: Redirecting users from legitimate websites to fake ones by exploiting DNS or host file manipulation.
- Phishing VS Pharming:
 - Phishing = tricking people directly (social engineering). Example: fake email or website asking you to log in.
 - Pharming = technical redirection. The attacker changes DNS or host settings so you visit a fake site even if you type the correct URL.
- Identity Theft: Unauthorized use of another person's personal data (e.g., SSN, credit card) to commit fraud or deception.
- Spam: Unsolicited bulk messages, usually advertisements, sent through email or messaging systems.

6. Ten Security Principles

No.	Principle	Summary
1	Least Privilege	Grant minimal access for minimal time.
2	Defense in Depth	Use multiple layers of protection.
3	Secure Failure	Systems should fail safely.
4	Secure the Weakest Link	A system is only as strong as its weakest part.
5	Compartmentalization	Isolate system components to limit damage.
6	Simplicity	Simple systems are easier to secure.
7	Promote Privacy	Protect user data diligently.
8	It's Hard to Hide Secrets	Avoid relying on obscurity.
9	Don't Extend Trust Easily	Be skeptical of all systems, even internal ones.
10	Trust the Community	Rely on well-tested, reviewed public solutions.

Security: Controls: who can do what when.

- Security protects information from unauthorized access, modification, or destruction.

Privacy: Freedom to control access to one's personal information.

- Privacy concerns how personal information is collected, used, and shared.
- Privacy is depending on intent
- In reality, we always trade privacy for service -> Medical, Social media

Security enables privacy, but a secure system can still violate privacy if data is used without consent.

- Hacker able to compromise a computer (unauthorized access -> its Security break) but if he sees some information owner don't want you to see then hacker break privacy (depend on owner mind)

AAA Security Model: Security Component

1. **Authentication:** Identify the user ("Who are you?")
2. **Authorization:** Verify permission ("What can you do?")
3. **Accounting/Auditing:** Track actions ("What did you do?") -> Known as the "AAA" or "Three-headed Kerberos".

Supporting Concepts

- **Integrity:** Consistency and wholeness of data. (Should be considered)
- **Input Validation:** "All input is evil until proven otherwise."
- **Threat Modeling:** Identify and mitigate system risks.

Password Security

- History:
 - Plaintext: Nothing
 - Hashed: One-way function
 - Salted: Adding constant (difference for everyone)
 - Adaptive Hashing (bcrypt, scrypt, Argon2): Hard to compute (designed for encrypt)
- Strength = function of length, charset, and guess limits.
- Weak passwords follow patterns (e.g., "Password1").

Password Hacking

- Dictionary: Use dict -> compute -> try
- Brute-force: Try every possible
- Rainbow table: Precomputed hashes for instant lookup.
- Replay: ตัดกับขโมย authentication ที่ได้ แล้วเอาขึ้นมาใช้ซ้ำ
- Social Engineer: Phishing

Defensive Measures

- Multi-factor authentication (SMS, authenticator apps).
- Strong password policies.
- Rate-limiting and lockouts.
- Use password managers.

Information Security Goals: CIA Triad

- **Confidentiality:** Only authorized users can access data.
- **Integrity:** Data must not be tampered or altered.
- **Availability:** Systems and data must be available to authorized users.

Auditing Concepts

- **Logging:** Record events or stats about system usage & perf.
- **Auditing:** Analyze logs to interpret system status clearly.

Auditing Goals

- User accountability: Known who's action
- Damage assessment: post-incident evaluation, analyze impact and improve future defenses
- Identify security violation causes: find root cause
- Describe system's security state: malware
 - Detect unauthorized system states
- Evaluate protection mechanisms
 - Deter attacks via audit records: Make hacker feel fears

What to log: Focus on data relevant to policy violations.

- Important event | Everything?

- Important -> might not see the problem
- Everything -> something might useless
- So choose everything that important instead -> policy violate (know exactly what going on)

- States vs. events

- States: Don't know what happen after recover
- Events: Know what happen

What to audit: Not all data, only policy-relevant info.

Log must record:

- Use of identification and authentication
- Object creation/deletion (e.g., file open)
- Admin/system actions
- Other security-related events

Each record includes:

- Date/time: timestamp
- User: user accountability
- Event type
- Success/failure
- Terminal ID (for auth) -> ip.hostname: identifies where a login or authentication request originated
- Object name (for file ops): what resource access

Link Layer Security

- Provides no confidentiality, integrity, or authentication. -> No security
- Attacks:
 - Sniffing: Monitoring
 - MAC spoofing: attacker manually changes their own MAC address. Other mechanisms (ARP, switching) behave normally but now treat the attacker as another host (possibly a trusted one).
 - Mainly attack: ที่ MAC address table switch เก็บไว้ self-learning
 - Self learning: Switch จะเก็บ <MAC, port> ลงมา port เน้น Switch MAC ที่รับได้
 - การส่ง frame ที่ไม่เจอ Switch MAC ที่รับได้ switch จะส่งออกมาหาเราแทน
 - ARP poisoning: broad term -> attacker sends forged ARP replies to corrupt other hosts' ARP tables (maps IP -> MAC wrong) -> Man In The Middle
 - ARP spoofing: a specific use of ARP poisoning where the attacker claims "IP X is at my MAC", tricking others into sending traffic to the attacker.
 - Mainly attack: IP table
 - ARP: เป็นตัว <IP, MAC> ที่ส่งออกไปส่งมา (ARP broadcast) แล้ว ARP response มาด้วย IP address
 - Gratuitous ARP: ส่ง IP มาหาเราโดยไม่พอ <IP, MAC> -> จักรวรรดิ ARP Response ปล่อย ส่งมา เพราะมันส่งมาทุกตัว
- **Defense:** secure switching (ARP), static ARP (ARP), encryption (Sniffing), Mac address randomization reduces tracking risk (MAC spoofing).
- New ARP -> to Switch เป็น Trusted ARP Responder (Proxy-ARP) หรือคือใช้ IP-MAC-Port จาก DHCP snooping อยู่แล้ว ดังนั้นเราแค่เสริมด้วย ARP ว่า "เราคือ IP X" -> ตัวต่อแบบนั้น โฉมใหม่แล้วจากเรา snooping แทนที่เราใช้ self-learning

Hardware mechanisms that support Integrity (isolation):

- **Ring levels (0-3)** for privilege separation.

- Ring 0: Kernel mode - full hardware access.
- Ring 3: User mode - restricted access.
- Inner can access outer ring. Outer ring can access inner through specific method (system call)
- Ring need -> 2 (Kernel, User) but mostly have 4, 1-2 for device driver

- **Segmentation:** Memory divided into logical segments (code, data, stack), access outside it is blocked.

- **Paging:** Memory divided into fixed-size blocks (pages) mapped by the OS to physical memory. Access control bits (read/write/execute) stop processes from writing into others' pages.

- **Segmentation + Paging** = isolate processes and prevent unauthorized access.

- **Tagged memory** Each memory word or block carries a tag describing its ownership or allowed operations.

Encryption Overview

- **Purpose:** Create Integrity & Security when no hardware support.

Classical history:

- Systeal (Sparta): Transposition cipher - สลับตำแหน่งตัวอักษร "ไม่เปลี่ยนแปลงตัวอักษรเอง"
- Caesar cipher (monoalphabetic): แทนแต่ละตัวอักษรด้วยตัวอักษรหนึ่งโดยใช้อุปกรณ์แบบแผนที่ (fixed mapping)
 - ตามที่เรารู้จักคือ Caesar Cipher คือ fixed shift ด้วยตัว a คือ 3 เช่น bad -> edg
 - แบบที่อาจารย์สอนคือ Caesar cipher คือ key shift ด้วยตัว a คือ 3 เช่น bad -> edg
 - abcdefgh... : standard alphabet
 - fodbacge... if key = "food" ข้าม f คือ food, key + rest of alphabet
 - bad -> ofa
- Weakness: Frequency analysis reveals plaintext patterns. -> เพราะในคำศัพท์อังกฤษ pattern การใช้อักษรมีค่อนข้างคงที่ เช่น e มักจะมากที่สุด

Stream Cipher: Encrypts bit-by-bit or character-by-character, extend key to match the size of input (how to extend based on algorithm).

- Depend On Pos: diff position & same char -> may encrypt differently
- **LUCKYLUCK** Lucky is key, extend to LUCKYLUCK to match input size
- **COMPUTING** Plain text
- **NIOZSECPQ** Cipher text
- Each pair <a,b> <b,a> will give same Char

Block Cipher: Encrypts fixed-size blocks, construct cipher blocks

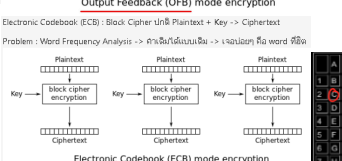
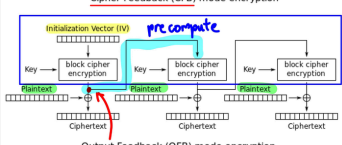
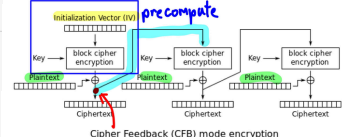
- Depend On Group: diff group & same char -> may encrypt differently
- **Fixed Size:** Every block must be the exact same size (e.g., 2 letters in Playfair).
- **The Constraint:** The system cannot process half a block.
 - **Odd Numbers:** Input has odd number of letters, add a dummy letter (e.g., X) to the end.
 - Example: **ROBOT** (5 letters) -> **RO BO TX**
 - **Double Letters:** Input a pair of identical letters in the same block. You must insert a dummy letter between them.
 - Example: **HELLO** -> **HE LX LO**

Variations:

- Initial vector: Vector ที่เอามา XOR กับ plaintext เริ่มต้น
- Padding: If block is not full -> use what (usually use 'X')
- Chaining: ผู้คนแต่ละบล็อกด้วย ciphertext ก่อนหน้า
- Feedback: ถ้า output จากบล็อกก่อน (plaintext หรือ output ของ encryption) มาใช้สร้าง key/IV ของบล็อกต่อไป -> ถ้า precompute ไม่ได้ ทำ output

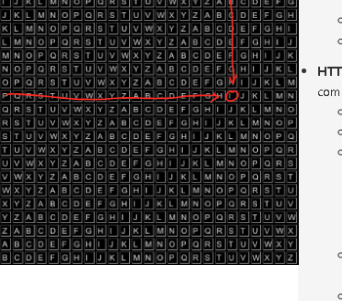
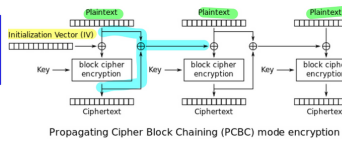
Example Scenario

Bob -> Alice
To ensure authenticity and confidentiality:
((MessageBob's PrivateKey)Alice's PublicKey ensures only Alice can read it (others like Bob's Private), and she knows it came from Bob).



IP Fragmentation

- Large packets: split and reassembled.
- Attacks:
 - Ping-of-death: สร้าง ICMP packet "ไม่บอกกินขนาดที่ reassemble แล้วทำให้เกิด overflow หรือ crash ในระบบ
 - Fragmentation-based DoS (resource exhaustion): ส่ง fragment จำนวนมาก กินคิวแล้วไม่ส่งจนทัน -> รอ reassembly ไม่เสร็จ เป็นโอ RAM/CPU -> DoS
 - Teardrop / Overlapping-fragment exploit: ส่ง fragment ที่ offset หนึ่งซ้อนกันอย่างไม่ดี ทำให้การรวมข้อมูลขึ้น เกิด buffer overflow หรือ crash ในระบบที่เฝ้าตรวจสอบไม่ถี่ -> อาจทำให้เครื่องล่มขึ้นได้
- **Defense:** packet validation, avoid fragmentation.



Network Layer (IP)

- **Best-effort:** no ordering, retransmission, or security.
- Attacks: IP spoofing, sniffing, DHCP abuse.
 - Routing (BGP)
 - Routers trust peers -> BGP hijacks (e.g., Pakistan-YouTube 2008) -> การหลอกทำ packet ไปขึ้นเป็นเราโดยที่ OS ถ้าไม่เจอ packet ก็จะส่งกลับไปยังที่ที่ MITM ถ้าส่งผิด
 - Validation: AS ต่าง ๆ จะเช็คกันเอง คือจะยืนยันให้ตัวเอง BGP -> ไม่มี authentication, ไม่มี validation.
 - **Defense:** Secure BGP (BGPsec)
 - **Concept:** ใช้ Cryptography เพื่อเชื่อมความสัมพันธ์ "ลูกโซ่" (Chain) เพื่อป้องกันการปลอมแปลง
 - **1. Chain of Trust (ทุกขบวนมาจากใน 2 - The Hierarchy):**
 - Router ไม่ได้เก็บเลขของ BGP (50,000+) โดยตรง
 - Router เก็บแค่ **Public Key ของ "Trust Anchors" (Trust Anchors - TA)** เช่น IANA/APNIC ไว้เท่านั้น
 - **Logic:**
 - Router เชื่อม **APNIC** (มี PubKey, APNIC)
 - **APNIC** เชื่อมกับ **Certificate_ASI** (ซึ่งไม่มี PubKey, ASI)

2. Example Scenario: (ส่งข้อมูลจาก AS1 -> AS2 -> AS3)

- **Step 1: AS1 (Origin) เริ่มต้น**
 - **Goal:** ประกาศว่า "ฉันเป็นเจ้าของ Prefix นี้ และฉันส่งต่อให้ AS2"
 - **Msg1** = "Prefix: 10.0.0.0/24, Target: AS2"
 - **Sig1** = PriKey_AS1(hash(Msg1))
 - **Packet Sent to AS2:** [Msg1, Sig1, Certificate_AS1]
- **Step 2: AS2 (Transit) รับมาและตรวจสอบ**
 - **Action 1: ตรวจสอบตัวตน (Identity Check - Chain of Trust)**
 - เห็น **Certificate_AS1**
 - Check: เห็น **PubKey_APNIC** (ที่ตัวเองมี) มาตรวจสอบ **Certificate_AS1** -> Check **PubKey_AS1** มาจากจากใครบ้าง
 - ใช้ **PubKey_AS1** และ **Sig1** ได้ **RcvHash**
 - True if **RcvHash** == **hash(Msg)**
 - **Result:** ข้อความถูกต้องจาก AS1 ซึ่งและจะส่ง Target เป็น AS2 จึง

- **Action 3: ส่งต่อ (Signing)**
 - **Msg2** = "Target: AS3"
 - **Sig2** = PriKey_AS2(hash(concat(Sig1, Msg2)))
- **Packet Sent: [Msg2, Sig2, Certificate_AS1, Certificate_AS2]**

- **Step 3: AS3 (Validator) เป้าหมาย**
 - **Action:** ต้องตรวจสอบย้อนกลับ
- **Layer 2 (ที่ AS2):**
 - 1. **Check Identity:** ใช้ **PubKey_APNIC** เช็ค **Certificate_AS2** -> "ได้ PubKey_AS2"
 - 2. **Check Sig:** ใช้ **PubKey_AS2** เช็ค **Sig2** -> ขึ้นต้นว่า AS2 ส่งให้ AS3
- **Layer 1 (ที่ AS1):**
 - 1. **Check Identity:** ใช้ **PubKey_APNIC** เช็ค **Certificate_AS1** -> "ได้ PubKey_AS1"
 - 2. **Check Sig:** ใช้ **PubKey_AS1** เช็ค **Sig1** -> ขึ้นต้นว่า AS1 ส่งให้ AS2

- **CuriosityCorner:**
 - **1. Why Encrypt Hash?** Encrypt(hash(Msg)) faster than Encrypt(Msg)
 - **2. Why concat(Sig1, Msg2)?**
 - If missing: Hacker can do a **Cut-and-Paste Attack** (เอา Sig2 ที่ถูกต้องไปแปะต่อท้าย Route ขึ้นไปปลอมขึ้น)
 - With Sig1: เมื่อมีการ "แปะเพิ่ม" เอกสารติดกัน ถ้า Sig1 (หน้าแรก) ถูกหักออกหรือเปลี่ยน ค่า Hash จะเปลี่ยน ส่งผลให้ Sig2 (หน้าสอง) ฟังงั้นที่ -> ขึ้นต้นว่าเส้นทางต่อไม่ขึ้นจริง (Broken Chain).

- **Comparison: ทำไม BGP Hijack ถึงทำได้?**
 - **Scenario:** Hacker (AS_Evil) ขโมยมาแทรกกลาง
 - **Attack:** Hacker รับ [Msg1, Sig1] จาก AS1 มา แล้วมาเปลี่ยนทางส่งให้ AS3
 - **Problem:**
 - Hacker ส่งต่อ **Sig_Evil** = PriKey_Evil(Sig1 + "Target: AS3")
 - แต่ตอน AS3 ตรวจสอบ AS3 จะเห็นว่า **Sig1** (ของ AS1) จะไป Target เป็น AS2 (ไม่ใช่ AS_Evil)
 - **Result:** Chain ขาด AS3 รุ่นที่ Hacker ไม่ได้รับอนุญาตจาก AS1 -> **Drop Packet!**

- **Defense:** packet validation, avoid fragmentation.
- **ICMP Abuse**
 - Can be used for redirection or reflection.
 - Smurf attack: ส่ง ICMP echo ไปยัง broadcast address พร้อมปลอม source IP ให้เป็นของเหยื่อ ผลลัพธ์: ทุกเครื่องตอบกลับ (ICMP Reply) ไปหาเหยื่อ -> Amplified DoS
- **Defense:** block directed broadcast packets.
- **Network Scanning and Fingerprinting**
 - Tools: Nmap for port/OS detection.
 - OS fingerprinting: OS แต่ละแบบมีลักษณะที่แตกต่างกัน จากการตอบกลับแบบที่ต่างกันเป็น OS ใด
 - จะใช้เพื่อหาจุดอ่อน

Security Protocols

- Combine algorithms to leverage strengths (e.g., speed & scalability).
- **Digital Signature:** Hash + Public key = fast, verifiable authenticity.
 - Use to check **message didnot modify (integrity)** and **come from expected sender (authenticity)**
 - **PriKey(Hash(RecvMsg)) = Digital Signature**
 - **if Hash(ResendMsg) == PubKey(DigSig)? Trustable : Fake/Modify**
- **HTTPS/SSL/TLS:** Combines symmetric and asymmetric encryption for key exchange and secure communication.
 - ใช้ Asymmetric ส่ง Key ของ Symmetric
 - ใช้ Symmetric ส่ง content
 - Root Cause:
 - Symmetric encryption ต้องใช้ "ชิ้นเดียวกันทั้งสองฝั่ง" -> ปัญหาคือ จะเอาชิ้นนี้ไปให้กันยังไงโดยไม่ให้โดนดัก?
 - ถ้าส่งไปตรง ๆ -> คนดักระหว่างทางก็จะเห็นชิ้นนี้ทั้งหมด -> ไม่ปลอดภัย
 - Asymmetric encryption แก้ปัญหา
- Conclusion: ใช้ Asymmetric เพื่อส่งชิ้นของ Symmetric อย่างปลอดภัย แล้วใช้ Symmetric เพื่อเข้ารหัสข้อมูลจริงในใจ
- **How we check the public key? -> Chain of trust** which will explain in PKI

System	Function	Limitation
Firewall	Filters IP/port headers	Cannot inspect payloads
IDS	Detects and logs suspicious packets (header+payload)	logs/alert not block
IPS	Detects and blocks packets (header+payload)	Can drop legitimate traffic

TCP/IP's design limits firewall accuracy (fragmentation, spoofed src/dst).

DNS attacks:

- Application-Layer protocol
- Flow การทำงานทั่วไป
 - ผู้ใช้งานขอชื่อโดเมน ผู้ให้บริการ URL เช่น www.chula.ac.th ในเบราว์เซอร์ → เครื่องจะต้องหาว่าโดเมนนี้มี IP Address อะไร
 - ระบบปฏิบัติการหรือเบราว์เซอร์จะขอ DNS cache ในเครื่อง (local) ก่อน
 - ถ้ามีข้อมูล (ถ้าไม่หมดอายุ TTL) จะใช้ IP นั้นทันที
 - else -> ถาม DNS Resolver (ของ InternetServiceProvider หรือองค์กร)
 - Resolver ตรวจสอบลำดับขั้นของ DNS
 - ตรวจสอบใน cache
 - ถ้ายังไม่รู้จักถาม Resolver จะทำการค้นแบบ "recursive"
 - ถาม Root DNS Server → ได้ชื่อของ Top-Level Domain (TLD) เช่น .th
 - ถาม TLD Server (.th) → ได้ชื่อของ Authoritative Server ที่ดูแล chula.ac.th
 - ถาม Authoritative Server (chula.ac.th) → ได้ IP ของ www.chula.ac.th
 - Resolver ส่งผลกลับให้เครื่องผู้ใช้งาน
 - Resolver จะเก็บผลไว้ใน cache เพื่อใช้ในอนาคต
 - ผู้ใช้งานได้ IP เช่น 161.200.192.4
 - นำ IP ที่ได้ไปเปิดการเชื่อมต่อผ่าน TCP หรือ HTTPS
- ปัญหา:
 - รับมี part additional ที่ใส่ <name>ip> ตอน authoritative ส่งคืนให้ DNS resolver
 - DNS resolver เชื่อสิ่งที่ DNS Server ส่งกลับมา 100%
- Spoofing (Kaminsky): DNS Spoofing -> เป็นวิธีการ
 - ตอน DNS resolver ถาม DNS Server -> ทำไฟล์ Cache Poisoning
 - เมื่อก่อนใน Response, port ส่วนใหญ่ที่เป็น 53 ดังนั้นเดาแต่ queryId ก็พอ ซึ่งแค่ 16 bits เท่านั้น เหมือนอิงตาม Birthday paradox โอกาส Q=256.R=256 แล้วถูกถึง 63 %
 - พยายามหาทำ DNS resolve ของเครื่องเป็นค่าส่วยแรง เรียกว่า ROP (Return-Oriented Programming)

$$P_{fail} = \frac{65,536 - 256}{65,536}$$

- เราปลอมไป 256 บิต (R = 256), lower_bound ต่อให้ R มีค่า

$$P_{256fail} = \left(\frac{65,536 - 256}{65,536}\right)^{256} = 0.367$$

$$P_{success} == 1 - 0.367 = 0.63$$

Defense:

- Source QueryId Random (16 bits -> 32 bits) -> แตกยากเพราะต้องแก้ Protocol
- increase-port randomization: เปลี่ยนจาก fixed ทำให้ออกมาเพิ่มเป็น 2^32 bits
- Cache poisoning
 - ผู้ใช้งานส่ง DNS response ปลอม ที่มี Additional Record แฉกโดเมนอื่นเข้ามาด้วย (ส่งด้วย DNS Spoofing)
 - ตัวอย่าง: ผู้ใช้งาน IP ของ a.com ส่งให้โดเมนแบบปลอม b.com = 6.6.6.6 มาด้วย ซึ่งจริงๆ b.com = 1.1.1.1, 6.6.6.6 เป็น ปลอมพอๆ
 - Defense: แก้ด้วยไม่ให้แฉก แต่มันก็จะทำให Perf แย่ จึงเปลี่ยนเป็นแฉกได้เฉพาะ web ที่อยู่ใน domain เดียวกัน (Bailiwick Checking) เช่น ns.a.com, mail.a.com

- Rebinding: เปลี่ยน IP ที่ผูกกับชื่อโดเมนให้เป็น Internal ip, TTL อันแรกน้อย รอบ 2 ส่งให้เป็น Internal IP แล้วให้ JS ที่ส่งไปจับข้อมูล

- DNS Spoofing ไม่ใช่ Required Condition: ถูกต้องครบ (ปกติแต่หลอกให้กดลิงก์ก็พอแล้ว)
- แต่ DNS Spoofing เป็น Catalyst (ตัวเร่ง): ช่วยบังคับให้คนที่ไม่ยอมกดลิงก์ก็แปลก กลายเป็นเหยื่อของ DNS Rebinding ได้
- Defense:
 - ส่งเบราว์เซอร์: บังคับป้องกัน DNS rebinding, refuse mid-session IP switch
 - ส่งเครื่องช่วย/Resolver: บังคับการ resolve ชื่อสาธารณะให้เป็น private IP ranges
 - Firewall: ป้องกันการเข้าถึงพอร์ตจัดการจากเครือข่ายภายนอก

- DNSSEC (auth & integrity) -> เพิ่ม "ลายเซ็นดิจิทัล" (Digital Signature) -> แก้ DNS Spoofing -> แก้ Cache Poisoning

TCP Vulnerabilities

- Spoofing:** Guess sequence number (1/2³¹ chance). ผู้ใช้งานสิ่งที่จะเกิดปลอมที่แสดงตัวเป็นเครื่องหนึ่งของเครื่องต่อ เพื่อแทรกข้อมูลหรือ hijack stream.
- Reset attack:** Inject TCP RST to terminate connection. ส่งแพ็กเก็ตที่มี seq RST เพื่อสั่งให้เครื่องส่งบิตการเชื่อมต่อทันที. ส่ง 3-4 tuple (srcIP, srcPort, dstIP, dstPort) และ sequence number ที่อยู่ใน window
- SYN Flood:** ส่ง SYN จำนวนมาก (มัก spoof source IP) ทำให้เซิร์ฟเวอร์สร้าง state (half-open) ไร่ ACK ที่จะไม่มา -> จ้าง backlog เต็ม ไม่รับการเชื่อมต่อใหม่จากผู้ใช้งาน
 - Good fixes:** SYN cookies, ไม่ใช่ backlog แต่ encrypt แล้วใช้คีย์มาส่งกลับตัวรับ.
 - With SYN cookies, server does not store state. Instead, it encodes connection info (client IP, port, timestamp, etc) inside the SYN-ACK's sequence number.
 - If the client is real, it replies with an ACK that includes this value.
 - Bad fixes:** Increasing queue or shortening timeout.

Asymmetric Encryption

- Keep **private key** secret; share **public key** freely.
- One key pair per person -> good scalability.
- Use cases:
 - Confidentiality** (encrypt with public key) -> only private key can read
 - Integrity** (verify with public key) -> digital signature
 - digital signature: hash(received message) ต่อหน้ากับ public(DigSig)

3. Missing Link – Authentication

Even with public keys, we cannot prove ownership without binding a key to an identity. -> ไม่รู้ว่าเป็นของใครเพราะในตัวของ public key ไม่มี identity signature -> ทำให้ public key นั้นน่าเชื่อถือผ่านคนกลางที่เข้าเชื่อถือ (Certificate Authority)

Digital Certificate

- Binds an entity's **public key** to its identity attributes.
- Issued and digitally signed by a trusted third party (CA).
- A ส่ง publicKeyA ให้ CA แล้ว CA ให้ Digital Certificate ให้ซึ่งมี privateKeyC(Cert,publicKeyA)
- B ที่ถือ CA หรือคนที่มี publicKeyC A ส่ง Digital Certificate ให้ B decrypt ก็จะเห็น publicKeyA และ Cert ที่ CA ออกให้ A -> น่าเชื่อถือ

5. Trust Model – Chain of Trust

If a trusted entity signs a certificate, you can trust the certificate and its subject.

Example chain:

Digicert Inc (Root CA) – Thawte TLS RSA (CA) – www.chula.ac.th

6. Anatomy of a Certificate

- Issuer: ให้ออกให้
- Subject: เจ้าของของใบรับรอง
- Subject Public Key: PublicKey
- Issuer Digital Signature: ลายเซ็น ที่ Issuer ใช้ private key เช่น certificate ของ Subject -> ใช้ตรวจว่าใครเป็นผู้ Issue (chain ได้มาจาก root server)

PKI Lifecycle

- Enrollment: ลงทะเบียนขอใบรับรอง
- Issuance: ออกใบรับรอง
- Validation: ตรวจสอบความถูกต้อง

1. Encryptio

Type	Strengths	Weaknesses
Hash/Digest	Fastest, ensures integrity	No confidentiality
Symmetric Encryption	Fast, provides confidentiality	Poor scalability (n เลือก 2 keys needed), integrity uncertain, no non-repudiation

Asymmetric Encryption	Confidentiality, integrity, scalable (2n keys needed), non-repudiation	100–1000× slower than symmetric encryption
-----------------------	--	--

Combination of these methods can solve most issues **except authentication**.

Anyone can create and sign their own certificate. Trust depends on manual verification by the recipient.

Self-Signed Cert

Root CA เป็นของตัวเอง

8. Legal and Operational Issues

เชื่อมโยง Infrastructure เพราะมีการนำ real-world มาช่วยกำกับ ไม่ให้ทำทุกอย่างจนผิด digital world

PKI Parties and Roles

- CA (Certificate Authority):** key generation, issuance, revocation, backup, cross-certification: ทำ Cert ให้
- RA (Registration Authority):** verify identity (face-to-face or remote), revoke requests: ตรวจสอบขอใบรับรองก่อนจะส่งให้ CA ทำ
- VA (Verification Authority):** ตรวจสอบใบรับรองก่อนจะออกใบรับรอง
- Certificate Distribution System:** certificates + CRLs (typically via LDAP):
 - ถ้ามีการดาวน์โหลดตรวจสอบใบรับรอง ของผู้ให้บริการอื่นไว้แล้ว
 - เมมเบร์ CRL เพื่อไม่ให้ถูกผู้รับใบรับรองใดถูกยกเลิกแล้ว
- Other components:** VA, policy database, PKI-enabled apps

Buffer overflow occurs when data exceeds buffer limits and overwrites **adjacent memory**, potentially altering control flow and executing malicious code. **Secure Bit** is a hardware-based mechanism designed to prevent such attacks by tracking whether input data is trusted.

3. Buffer Overflow Works

Example in C using `gets()` -> overwrites stack memory. 'l' = 0x31 = 49 **Attack types:**

- Return address modification (stack smashing) = เราเขียนจนเกิน Return Address ทำให้มันไปรันอะไรก็ได้. เราเอาโค้ดมาเปลี่ยนให้มัน Return กลับมาขึ้นยังได้
 - Mandatory Condition
 - 1. Input แปลกเขียนได้
 - 2. Address สำคัญ (Return address) โดเมน
 - Sol: ไม่ให้เกิด 2 conditions พร้อมกัน
 - Input Range/Value Validation : ของที่เราให้เขียนได้เท่าขึ้น
 - Critical Code Validation : ดูว่าถูกแก้ไขไหม -> digital signature

Static Analysis (Lexical/Semantic Analysis)

- Lexical (ดูคำศัพท์): ตรวจสอบจาก code อย่างค่าแบบ strcpy มีโอกาสสูงทำให้เกิด buffer overflow
 - Tools: ITSA(String Matching), FlawFinder, RATS
- Semantic (ดูความหมาย/บริบท): ตรวจสอบ "ตรรกะ" ถูกประกาศตัวแปรไว้กี่ครั้งแล้ว แต่ต้นเอาข้อความไปใส่ -> "ผิดประเภท"
 - Tools: Splint, BOON
- Pros: Detects known issues pre-deployment; ค้นหาคำผิด
- Cons: No runtime protection (ไม่ช่วยตอนรันแล้ว), false positives (Warning เมมเบร์เกิน)

4.3 Isolation

- Non-executable memory (NX), sandboxing
 - Memory บางส่วนไม่อนุญาตให้ execute
 - ไม่ใช่ root cause เพราะ NX ก็มีการขัดโค้ดในรันไทม์ แต่ยังไม่ใช้คำสั่งที่มีอยู่แล้วในเครื่องได้ ขัดติดขัดต่อกันจนกลายเป็นคำสั่งร้ายแรง เรียกว่า ROP (Return-Oriented Programming)

4.4 Secure Boost

- hash กับ kernel code ไว้เป็น digital signature -> ถ้าไม่ตรงไม่ boost

5. Theory

Definition: Buffer overflow happens when external input modifies addresses. **Theorem:** If an address cannot be modified (or modification can be detected), buffer-overflow attack is impossible. **Corollary:** Preserving address integrity prevents buffer overflow.

7. Secure Bit Concept

Idea: Tag all data from untrusted domains with a "Secure Bit."

- When data crosses domain boundaries, the bit is set.
- CPU disallows such data as jump/control targets.
- Ensures "untrusted input ≠ executable control."

File Carving

- Reassemble deleted or fragmented files from disk space.
- Computer don't immediately remove data that is deleted.
 - After delete the original data is still present, but marked as unallocated space, even some or all of the data has been overwritten, the remaining data can still be carved and reviewed.

Threat	Mitigation
Spoofing (ปลอมตัว)	Authentication
Tampering (แก้ไข/ปลอมแปลง)	Integrity
Repudiation (ปฏิเสธการรับผิดชอบ)	Non-repudiation
Information Disclosure (เปิดเผยข้อมูล)	Confidentiality
Denial of Service (ไม่ทำงาน)	Availability
Elevation of Privilege (เพิ่มสิทธิ์)	Authorization
Disable unused ports/services.	
Turn off UDP (if your service does not need)	
Restrict network ranges. (only allow for trusted network)	
Authenticating connections.	
Harden ACL (Access Control Lists). (Admin = full access, Everyone = read only, Service = Read&Write)	
Run services with least privilege.	

Secure Design Principles

- Plan security from the start, not after implementation.
- Attack Surface Reduction (ASR):**
 - Attack Surface: Where attacker can enter or extract data from a system
 - Less code running = fewer vulnerabilities
 - Defense in Depth – multiple layers of protection -> remove single point of failure
 - Least Privilege – limit permissions
 - Secure Defaults – turn off unnecessary features -> less stuff to attack

ทำกัน Capability เป็นเชิงรุกหาจุดอ่อนให้ดีกว่า?

- ACL:** ระบุผู้เข้า "The Confused Deputy" (ผู้รับเกิน)
- ขอบเขต ACL:** เจตนาว่า "ไม่ยอมให้ใครทำอะไร" (เช่น เป็นแฉก)
- ขอบเขต:** หลอกใบรับรองได้ "ไม่ยอมให้ใครทำอะไรจากข้างในให้เราได้"
- Capability:** แก้ไขเพราะไม่ยอมให้ใครทำอะไรด้านใน ทำให้ได้ User's Discretionary Access Control (DAC)

Access Control Matrix

- Access Control Lists (ACL)** -- object-centric, Popular
 - Each user/role can do what to this resource, attached to Resource
 - ของ (resource) ที่เพิ่มหรือลบเฉพาะว่าใครทำอะไรกับของนี้ได้บ้าง
- Capability-based System** -- subject-centric, like tokens/keys
 - This user/role can do what to each resource, attached to Subject
 - บุคคล (user/role) ที่เพิ่มหรือลบเฉพาะว่าตัวเองทำอะไรกับของนี้ได้บ้าง

Security Models

define Policy (Blueprint) -> Guideline

- Multilevel Security: Within Org, MAC
 - Bell-LaPadula (Confidentiality): Focus = Secrets
 - No Read Up (NRU), No Write Down (NWD)
 - Prevents information leaks.
 - หน่วยงานราชการ, กองทัพ ที่ "ความลับ" สำคัญที่สุด
 - Biba (Integrity): Focus = Trust/Truth
 - No Read Down, No Write Up
 - Prevents corruption from lower-integrity sources.
 - ธนาคาร, ระบบบัญชี ที่ "ความถูกต้อง" สำคัญที่สุด
- Multilateral Security: China Wall Model
 - Conflict of Interest: Dynamically blocking access to competitor data.
 - สำนักงานราชการ, บริษัทมหาชน, ตลาดหลักทรัพย์ เพื่อป้องกันข้อมูลภายในขององค์กร
 - เอาข้อมูลขององค์กร A ไปปนกับองค์กร B

Address Protection:

- Canary Words: มาจาก Canary Bird (นกขมิ้น->อ่อนแอ ตายง่าย) -> สะดุดก่อน RET โจมตี
 - กั้นระหว่าง Buffer กับของสำคัญๆ ถ้า overflow Canary ต้องถูกแก้ไข
 - ถ้ามีค่าเป็น "Null Byte" (0x00) อยู่ด้วย เพราะ **strcpy** หยุดทำงานเมื่อเจอ Null Byte
 - ไม่ใช่แค่ใส่สัญญาณเตือน แต่ยังเป็น "กำแพง" ที่ทำให้ **strcpy** หยุดทำงานด้วย
 - Stack Guard:** Canary word before return address -> Detect Overwrite
 - ปัญหา: ถ้า hacker ใช้ Canary Word ที่แก้ไขแล้วไม่ถูกก็พอ
- Address Encode: Encrypt Address ทำให้ hacker แก้ได้ แต่เนื่องจากไม่รู้ว่า Encrypt อย่างไร เรายัง Decrypt ก็จะไม่ทันหน้าผู้ใช้ ซึ่งมีโอกาสสูงจะ Crash -> ทำให้เราไม่ไปรันโค้ดที่ hacker เอาไว้ให้
- PointGuard:** Encrypts pointers. Protects function pointers
- SPEF:** Encrypted instruction/code
- Instruction Set Randomization (ISR):** เข้ารหัส/ชุดคำสั่งด้วย per-process key (เช่น XOR) ทำให้โค้ดที่โจมตีเขียนมาไม่ตรงกับ ISA ที่ถูกถอดรหัสจริง (Columbia / Drexel)
 - ปกติ ADD อาจจะเป็น Opcode 0x01 ในทุกที่เจอ.
 - ISR จะสุ่มให้ Process A ใช้ ADD = 0x55, Process B ใช้ ADD = 0xAA, ซึ่ง hacker เค้าไม่รู้
- ปัญหา: Performance, Compatibility
- Copy of Address: เก็บ return addr สองที่จริง, เวลาจะ ret จะเก็บสองค่าถ้าไม่ตรงก็ดูอีกเขียนทับ
 - Split Stack:** Separates control(ret)/data(buffer) stack, มี ret ที่เดียว แยกไว้
 - ปัญหา: ทับสองที่ที่โดนอยู่

- Tags: บิดพิเศษ meta data ที่ติดไปกับแต่ละคำของหน่วยความจำ (memory word) เพื่อบอกว่าให้แสดงว่า "ข้อมูลนี้มาจากที่มาใด" ถ้าจะใช้ทำเป็น Addr แด่ต้นมาจาก input -> "ไม่อนุญาต"

- Input Protection:** Prevents untrusted data as control data -> Secure bit

Bounds Checking: Validates memory access: ของที่เราให้เขียนเท่านั้น — ทางคนที่ถูกส่งแต่แพง

- Array Bounds Checking: Software-level checks
- Segmentation / hardware bounds: Hardware-level checks
- LibSafe / LibVerify:** พ่วงไลบรารี เพื่อตรวจสอบ bounds และการจัดการ I/O/strcpy กันเขียนจริง (Bell Labs)
- ปัญหา: ข้างล่างช้ามาก (ตัวอย่างในเอกสารอ้างถึง ≈30x slowdown ในบางกรณี)

Obfuscation: ASLR (Address Space Layout Randomization) -> เค้าไม่รู้ว่า RET อยู่ไหน, Hack ได้แต่ยากขึ้นเฉยๆ จากเมื่อก่อนแค่อ่าน Source Code ก็ทำได้แล้ว

4. Data Hiding Techniques : Disguises information from normal view

- Rename:** Changing name and extension -> make it look harmless/irrelevant
 - Weakness: Forensic tool check file header can compare the extension
- Attributes hiding:** Mark as Hidden in OS -> disappears from normal file explorer view
 - Weakness: Visible if system show hidden files
- Bit-shifting:** Shift binary data to hide content -> file become unreadable (lightweight encryption) -> but easy to crack.
- Encrypt/Password:** Data unreadable without key
- Partition hiding (ไม่ mount): diskpart remove letter** -> OS won't display that partition (OS wont mount), though data still exists.
 - Weakness: Visible to forensic tools that scan unallocated space, analyze disk space
- Marking bad clusters (Mount เค้าไม่ให้เพราะถูก mark ว่าพัง):** FAT file system, attacker can manually mark good clusters as "bad", so the OS skips them, assuming they are damaged.
 - Weakness: Detectable with modern forensic tools but invisible to casual users.

Steganography & Steganalysis

- Steganography:** hiding information inside other files (images, audio).
 - เอาข้อความลับซ่อนไว้ในรูปภาพ โดยเปลี่ยนค่า บิตเล็ก ๆ (LSB – Least Significant Bit) ของพิกเซล
 - ส่งข้อความในไฟล์เสียง/วีดีโอโดยเปลี่ยนค่าที่มนุษย์ฟังไม่ต่าง
 - Keys: Human not noticable to these changes
- Steganalysis:** detecting and analyzing Steganography.
- Digital watermarking:** hides ownership info -> บอกว่าไฟล์นี้ของใคร, พยายามเอาข้อมูล. Steganography that for license purpose not hiding information เช่น ข้างภาพซ่อนชื่อใน .jpg
- Authorization:** "What can you do?" -> **Policy will tell you**
 - Policy (Law):** Help to decide
 - App-specific decision-making based on policy : Something action is different in different application -> No general AuthZ can use for every application
 - Type Enforcement (Police): Enforce to follow policy

- Security Policy:** Clearly define what is secure or insecure. We cannot say something is "safe" unless we define what "safe/unsafe" looks like
- Secure System:** Start Secure + No Insecure Moves = Always Secure.
 - Starts in a secure state
 - Cannot enter an insecure state.

Access Control Models

- Mandatory Access Control (MAC)
 - Admin-defined policies; users untrusted.
 - Multilevel system : Higher Level is king.
- Discretionary Access Control (DAC)
 - Owner-defined policies
 - Common in personal computing/drive.
- Role-Based Access Control (RBAC)
 - Who know about role defined policies
 - Based on least privilege; access tied to job roles.
 - No clear definition on how to define a role lead to **Role Explosion**
 - "Nurse" role.
 - "Nurse who works night shift" role
 - "Nurse who works night shift and handles X-rays."
 - Suddenly, you have 1,000 roles, and it's a mess.
- Attribute-Based Access Control (ABAC)
 - Decisions use attributes to map access ability.
 - Attributes : User attr, Resource attr, Environment attr
- Modern Access Control
 - OAuth 2.0 / OIDC:** Limit-Accessed Pass
 - Zero Trust Architecture:** Never Trust, Always Verify

Policy Development Steps

- Analyze security model (Bell-LaPadula, Biba, China Wall).
- Choose access control model (MAC, DAC, RBAC, ABAC).
- Convert policy – Access Control Matrix.
- Implement via Type Enforcement (ACLs, Capabilities).